

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»

Группа Р3211

Дисциплина: Вычислительная математика

ЛАБОРАТОРНАЯ РАБОТА №1

по теме: «Решение системы линейных алгебраических уравнений
СЛАУ»

Вариант 11

Выполнил:
Михайлов Петр Сергеевич

Преподаватель:
Малышева Татьяна Алексеевна

Санкт-Петербург, 2026
03.03.2026

Содержание

1	Цель работы	3
2	Описание метода и расчетные формулы	4
3	Листинг программы	5
4	Примеры и результаты работы программы	7
5	Выводы	10

1 Цель работы

Практическое освоение прямых методов решения систем линейных алгебраических уравнений (СЛАУ) путем программной реализации классического метода Гаусса.

В рамках лабораторной работы необходимо:

- Реализовать численный метод в виде изолированного модуля (класса), принимающего исходные данные в качестве параметров.
- Обеспечить возможность ввода размерности матрицы ($n \leq 20$) и ее коэффициентов как с клавиатуры (через пользовательский интерфейс), так и из файла.
- Вывести промежуточные и итоговые результаты: треугольную матрицу (включая преобразованный столбец свободных членов B), вектор неизвестных $x_1, \dots, x_n$ и вектор невязок $r_1, \dots, r_n$.
- Реализовать вычисление определителя по полученной треугольной матрице.
- Решить различные СЛАУ и найти определитель с использованием готовых математических библиотек, после чего сравнить результаты с собственной реализацией и объяснить их сходство или различие.

2 Описание метода и расчетные формулы

Метод Гаусса относится к прямым (точным) методам решения СЛАУ. Суть классического метода заключается в последовательном исключении неизвестных для приведения расширенной матрицы системы к верхнетреугольному виду.

Процесс решения состоит из двух этапов: прямого и обратного хода.

Прямой ход (приведение к треугольному виду)

На данном этапе матрица системы приводится к верхнетреугольному виду с помощью элементарных преобразований строк. Пусть дана система $Ax = b$. На k -м шаге исключения переменной x_k из нижележащих уравнений коэффициенты пересчитываются по формулам:

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} a_{kj}^{(k-1)}, \quad i, j = k+1, \dots, n \quad (1)$$

$$b_i^{(k)} = b_i^{(k-1)} - \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} b_k^{(k-1)}, \quad i = k+1, \dots, n \quad (2)$$

где $a_{kk}^{(k-1)} \neq 0$ — ведущий элемент.

Обратный ход (вычисление неизвестных)

После приведения системы к треугольному виду неизвестные последовательно вычисляются снизу вверх (начиная с x_n до x_1) по формуле:

$$x_i = \frac{b_i^{(i-1)} - \sum_{j=i+1}^n a_{ij}^{(i-1)} x_j}{a_{ii}^{(i-1)}}, \quad i = n, n-1, \dots, 1 \quad (3)$$

Вычисление определителя

Важным свойством метода Гаусса является то, что определитель исходной матрицы равен определителю полученной треугольной матрицы. Определитель треугольной матрицы вычисляется как произведение элементов на её главной диагонали:

$$\det(A) = \prod_{i=1}^n a_{ii}^{(i-1)} = a_{11} \cdot a_{22}^{(1)} \cdot a_{33}^{(2)} \cdot \dots \cdot a_{nn}^{(n-1)} \quad (4)$$

Вычисление вектора невязок

Для проверки точности найденного решения x вычисляется вектор невязок r . В идеальном случае (при точной арифметике) он равен нулевому вектору. Формула для вычисления вектора невязок:

$$r = Ax - b \quad (5)$$

или поэлементно: $r_i = \sum_{j=1}^n a_{ij} x_j - b_i$.

3 Листинг программы

Программный комплекс реализован в виде полноценного клиент-серверного веб-приложения. Общение между клиентом и сервером происходит по протоколу HTTP (REST API).

Выбранный стек технологий:

- **Backend (Серверная часть):** Написан на языке **Python 3.14** (отлично подходит для математических вычислений). В качестве каркаса используется высокопроизводительный фреймворк **FastAPI**, который «из коробки» генерирует документацию к API (Swagger). Для строгой валидации входящих данных (матриц и векторов размерностью $n \leq 20$) применяется библиотека **Pydantic**. В качестве эталонной реализации для сравнения результатов и определителя используется библиотека **NumPy**.
- **Frontend (Клиентская часть):** Реализована на **TypeScript** с использованием библиотеки **React** (инструмент сборки Vite). Интерфейс позволяет динамически генерировать формы для ручного ввода коэффициентов матрицы, а также поддерживает загрузку исходных данных из файла. Взаимодействие с сервером осуществляется посредством HTTP-клиента **axios**.

Численный метод изолирован в отдельном классе на стороне сервера. Ниже представлен листинг класса **GaussSolver**, реализующего метод Гаусса. С полным исходным кодом проекта (включая клиентскую и серверную части) можно ознакомиться в репозитории на GitHub: ссылка на репозиторий.

```

1 import numpy as np
2
3 class GaussSolver:
4     def __init__(self, matrix_a, vector_b):
5         self.a = np.array(matrix_a, dtype=float)
6         self.b = np.array(vector_b, dtype=float)
7         self.n = len(vector_b)
8         self.swaps = 0 # Для корректного знака определителя
9
10    def solve(self):
11        # 1. Прямой ход (приведение к треугольному виду)
12        for i in range(self.n):
13            # Проверка на нулевой ведущий элемент и перестановка
14            if abs(self.a[i][i]) < 1e-12:
15                pivot_found = False
16                for k in range(i + 1, self.n):
17                    if abs(self.a[k][i]) > 1e-12:
18                        self.a[[i, k]] = self.a[[k, i]]
19                        self.b[[i, k]] = self.b[[k, i]]
20                        self.swaps += 1
21                        pivot_found = True
22                        break
23                if not pivot_found:
24                    raise ValueError("Матрица вырожденная или имеет бесконечное множество реш
25                        ений.")
26
27                # Исключение неизвестных
28                for k in range(i + 1, self.n):
29                    c = self.a[k][i] / self.a[i][i]
30                    self.a[k, i:] = self.a[k, i:] - c * self.a[i, i:]
31                    self.b[k] = self.b[k] - c * self.b[i]
32
33            # Сохраняем треугольную матрицу (включая преобразованный B) для вывода
34            triangular_res = np.column_stack((self.a, self.b)).tolist()
35
36            # 2. Вычисление определителя
37            # Определитель треугольной матрицы произведение диагональных элементов
38            det = np.prod(np.diag(self.a)) * ((-1) ** self.swaps)
39
40            # 3. Обратный ход
41            x = np.zeros(self.n)
42            for i in range(self.n - 1, -1, -1):
43                s = np.dot(self.a[i, i + 1:], x[i + 1:])
44                x[i] = (self.b[i] - s) / self.a[i][i]
45
46            return {
47                "triangular_matrix": triangular_res,
48                "solution": x.tolist(),
49                "determinant": float(det)
50            }
51
52    def get_residuals(self, x_solution, original_a, original_b):
53        """Вычисление вектора невязок  $r = Ax - b$ """
54        a = np.array(original_a)
55        b = np.array(original_b)
56        x = np.array(x_solution)
57        #  $r_i = \sum(a_{ij} * x_j) - b_i$ 
58        residuals = np.dot(a, x) - b
59        return residuals.tolist()

```

Листинг 1: Класс GaussSolver (Backend на Python), реализующий алгоритм

4 Примеры и результаты работы программы

Для проверки корректности работы программы были проведены тесты на матрицах различной размерности: $n = 3$ и $n = 5$.

Размерность (n):

Матрица коэффициентов A

10	-7	0
-3	2	6
5	-1	5

Вектор B

7
4
6

Рис. 1: Ввод данных для матрицы размерностью 3×3

Результаты вычислений

1. Треугольный вид (Прямой ход Гаусса)

10	-7	0	7
0	-0.10000000000000009	6	6.1
0	0	154.99999999999999	154.99999999999986

* Последний столбец выделен цветом — это преобразованный вектор B.

2. Метод Гаусса

Определитель: -155.00000000000003

Вектор неизвестных (X):

$x_1 = -9.325873406851315e-15$ $x_2 = -1.0000000000000133$

$x_3 = 0.9999999999999998$

Вектор невязок (R):

$r_1 = 0$

$r_2 = -4.440892098500626e-16$

$r_3 = -3.375077994860476e-14$

3. Библиотека NumPy

Определитель: -155.00000000000003

Решение NumPy:

$x_1 = 0$

$x_2 = -1$

$x_3 = 1$

Рис. 2: Результаты вычислений для матрицы 3×3

Размерность (n):

5

Загрузить из файла

Матрица коэффициентов A

20	4	9	1	-1
1	25	-1	2	1
2	14	30	-1	1
1	-20	1	10	40
-1	1	30	44	-2

Рассчитать решение

Вектор B

33
28
77
35
42

Рис. 3: Ввод данных для матрицы размерностью 5x5

Результаты вычислений

1. Треугольный вид (Прямой ход Гаусса)

20	4	9	1	-1	33
0	24.8	-1.45	1.95	1.05	26.35
0	0	29.895161290322584	-2.1693548387096775	0.5241935483870969	59.25
0	0	0	11.492514162395468	40.916306986781755	56.0631912597788
0	0	0	0	-167.0142948518581	-243.34356350235606

* Последний столбец выделен цветом — это преобразованный вектор B.

2. Метод Гаусса

Определитель: -28461072.999999996

Вектор неизвестных (X):

$x_1 = 0.6403942325013539$ $x_2 = 1.1381922600036898$
 $x_3 = 1.9339452873052254$ $x_4 = -0.30913890702574703$
 $x_5 = 1.4570223687631176$

Вектор невязок (R):

$r_1 = -7.105427357601002e-15$
 $r_2 = -3.552713678800501e-15$
 $r_3 = -1.4210854715202004e-14$
 $r_4 = 2.1316282072803006e-14$
 $r_5 = -7.105427357601002e-15$

3. Библиотека NumPy

Определитель: -28461073

Решение NumPy:

$x_1 = 0.6403942325013539$
 $x_2 = 1.13819226000369$
 $x_3 = 1.9339452873052256$
 $x_4 = -0.30913890702574703$
 $x_5 = 1.4570223687631174$

Рис. 4: Результаты вычислений для матрицы 5x5

Сравнение с эталонной библиотекой NumPy

Анализ полученных результатов позволяет сделать вывод, что собственная реализация классического метода Гаусса успешно решает поставленную задачу. Вектор невязок для обеих тестовых систем близок к нулевому (компоненты вектора имеют порядок $10^{-14} \dots 10^{-16}$), что является отличным показателем точности.

Однако при детальном сравнении результатов ручной реализации с ответами библиотеки NumPy наблюдаются характерные микроскопические различия, природа которых заключается в особенностях машинной арифметики:

1. Накапливаемая погрешность округления вещественных чисел:

В тесте размерности $n = 3$ эталонная библиотека NumPy выдает точные целочисленные ответы: $x_1 = 0$, $x_2 = -1$, $x_3 = 1$. Наша реализация выдает значения:

$$x_1 = -9.325873406851315 \cdot 10^{-15}, \quad x_2 = -1.0000000000000133, \quad x_3 = 0.9999999999999998$$

Это различие вызвано тем, что в классическом методе Гаусса многократно выполняются операции умножения, деления (особенно деление на ведущий элемент a_{ii}) и вычитания. Тип данных 'float64' в стандарте IEEE 754 имеет ограниченную мантиссу (около 15-17 значащих десятичных цифр). С каждой новой операцией в младших разрядах накапливается погрешность. Библиотека NumPy внутри использует высокооптимизированные подпрограммы на C/Fortran (LAPACK/BLAS), которые могут применять более сложные стратегии выбора главного элемента или внутренне работать с повышенной точностью, а также аккуратнее форматировать финальный вывод.

2. Погрешность вычисления определителя:

Для матрицы $n = 3$ определитель в обеих реализациях совпал с точностью до последних знаков (-155.00000000000003). Для матрицы $n = 5$ реализованный метод выдал $\det \approx -28461072.999999996$, в то время как NumPy отобразил -28461073 . В нашем алгоритме определитель считается как произведение диагональных элементов треугольной матрицы $\prod a_{ii}$. Поскольку сами диагональные элементы уже были получены в результате серии вычислений с плавающей точкой в ходе прямого хода, их погрешности перемножаются, давая небольшое отклонение от "идеального" математического значения на 16-м знаке после запятой.

Таким образом, выявленные различия между ответами NumPy и собственной реализацией не являются ошибками алгоритма. Они служат наглядной иллюстрацией накапливаемой вычислительной погрешности при работе с числами с плавающей точкой в прямых численных методах.

5 Выводы

В ходе выполнения лабораторной работы был изучен прямой метод решения систем линейных алгебраических уравнений — метод Гаусса.

Был успешно реализован программный комплекс с клиент-серверной архитектурой (React + FastAPI), обеспечивающий:

- Ввод данных как через динамическую веб-форму, так и путем загрузки файлов конфигурации.
- Прямой ход метода Гаусса для приведения матрицы к верхнетреугольному виду.
- Вычисление определителя системы на основе диагональных элементов треугольной матрицы.
- Обратный ход для вычисления искомого вектора неизвестных.
- Вычисление вектора невязок для оценки точности полученного решения.

Реализованная программа показала свою работоспособность на матрицах размерности $n \leq 20$. Сравнение результатов самописного алгоритма с эталонным решением из библиотеки 'NumPy' подтвердило корректность логики. Выявленные в ходе сравнения расхождения в младших разрядах дробной части (на уровне 10^{-15}) успешно объяснены спецификой стандарта IEEE 754 и накоплением погрешности округления при делении и умножении вещественных чисел, что является нормальным поведением для численных методов. Вычисленный вектор невязок практически равен нулевому вектору, что свидетельствует о высокой точности найденного решения.